

A machine learning ensemble approach to predict financial markets manipulation

Tatiana Franus^a, Malvina Marchese^a, Richard Payne^a

^a*Bayes Business School, City University of London, 106 Bunhill Row, London EC1Y 8TZ, UK*

Abstract

Forecasting spoofing manipulation in financial markets is the cornerstone of regulators to improve trading surveillance systems. This paper presents a novel data-driven methodology for predicting market conditions associated with episodes of spoofing manipulation. Our approach diminishes the significance of model selection by combining forecasts from various conventional machine learning techniques. We implement the forecasting method using a distinctive dataset of suspicious spoofing orders identified on the Moscow Stock Exchange. Our research demonstrates that examining the limit order book and prior alleged spoofing events produces a reliable manipulation prediction metric for short-term intervals in a high-frequency data context. The Real-Time Spoofing Probability measure proposed is shown to be an effective real-time indicator of risk to trade in manipulative environment, which can be used by exchanges, market participants and regulators in surveillance systems.

Keywords: Price Manipulation, Spoofing, Forecast Combination, Random Forest, XGBoost, Ensemble

1. Introduction

Electronic markets and high-frequency automated trading have transformed the financial market landscape and engendered potential for disruptive behaviours. Such practices have the potential to enable traders to make considerable profits through the artificial manipulation of market perceptions, which, in turn, may detrimentally impact other market participants. The strategic manipulation of markets can lead to price distortions, posing a substantial threat to the trustworthiness

Email addresses: `tatiana.franus.2@city.ac.uk` (Tatiana Franus), `malvina.marchese@city.ac.uk` (Malvina Marchese), `richard.payne.1@city.ac.uk` (Richard Payne)

and integrity of capital markets. The resulting mispricing may undermine investor confidence and damage market participation, efficiency, liquidity and the overall development of financial markets (Punniyamoorthy and Thoppan, 2013; Imisiker and Tas, 2013; Guiso et al., 2008).

One prominent form of trade-based manipulation is 'spoofing'. Spoofing involves the placement and subsequent withdrawal of limit orders to feign supply or demand interest. In spoofing strategies, manipulators place spurious orders in the order book to pose as traders with legitimate buy and sell interests. These orders are not intended to be executed but are designed to mislead other market participants with artificial volume. In the aftermath of the enactment of the Dodd-Frank Act, which outlawed such activities (Dodd-Frank, 2010), substantial investments have been made in the enhancement of automated surveillance systems tasked with its detection. The Commodity Futures Trading Commission (CFTC) responded to the threat posed by spoofing with the establishment of a dedicated Task Force in 2018, signaling the critical importance of thwarting such manipulation for regulatory bodies. Notably, financial institutions such as Citigroup and J.P. Morgan Chase have been subjected to sizable fines amounting to \$25 million and \$920 million, respectively, for engaging in spoofing within the US Treasury futures market between the years 2017 and 2020.

The threat of manipulation and the necessity to police it has prompted the emergence and rapid expansion of a trade surveillance industry dedicated to the oversight of client transactions and the identification of illicit trading activities (Aitken et al., 2015; Cumming et al., 2011). However, these surveillance systems still lack effective tools for predicting spoofing activities — a challenge that impedes regulators' real-time monitoring efforts.

This paper makes four innovative contributions to the literature on spoofing, a trade-based manipulation. First and foremost, We initially create an innovative data-driven methodology predict the market conditions linked to episodes of spoofing manipulation. Second, we identify the link between market states and limit order book variables. Third, we provide a comprehensive and methodical assessment of the out-of-sample efficacy of several machine learning algorithms utilised for spoofing detection. Finally, we present an efficient spoofing prediction metric, the Real-Time Spoofing Probability (RTSP), derived from the combination of supervised machine learning algorithms. Our approach is assessed utilising a distinctive and innovative dataset of suspected spoofing incidents recorded by the Moscow Exchange (MOEX).

Over the past ten years, there has been a growing interest in employing machine learning

for the detection of market manipulation. Ögüt et al. (2009) examine the efficacy of Support Vector Machine (SVM) and Artificial Neural Networks in detecting manipulation on the Istanbul Stock Exchange. Golmohammadi and Zaiane (2015) advocate for contextual anomaly detection algorithms and demonstrate their superior performance over k-Nearest Neighbor techniques (kNN) in a broad stock market manipulation identification. Cao et al. (2014) utilise various machine learning techniques to identify manipulation in financial markets. Their outcomes suggest that the kNN and SVM methods are capable of detecting manipulative behaviors with a notable degree of effectiveness. Tuccella et al. (2021) employ a Gated Recurrent Unit (GRU) architecture for spoofing uncovering across various cryptocurrency exchanges, whereas Tao et al. (2022) introduces a measurement instrument to assess real-time spoofing behaviour in the Toronto Stock Exchange utilising a conditional Wasserstein distance. Cao et al. (2015) assess the efficacy of the Adaptive Hidden Markov Model in comparison to conventional methodologies, such as Gaussian Mixture Models and k-Nearest Neighbours, to detect manipulative trading activity on the London Stock Exchange and NASDAQ. The findings from their study indicated an improved detection capability.

All these studies suffer from certain limitations. First, they detect spoofing orders via a technique they developed, which raises concerns regarding the efficacy of the detection approach. In contrast, we identify spoofing using suspected spoofing incidents recognized by exchange authorities. Moreover, most studies rely on synthetically generated data only to validate their findings. We rely on a comprehensive dataset containing entire historical records of limit and market orders. In our empirical research, we construct a unique dataset by combining data of all orders and transactions alongside data within the same timeframe from MOEX with suspected spoofing orders detected by the exchange’s proprietary algorithm across ten liquid stocks. The selected ten stocks are not the most actively traded on the Moscow exchange, aligning with the findings of Williams and Skrzypacz (2020), which indicate that spoofing is most widespread in markets that possess adequate liquidity without being excessively liquid. By utilising files including all orders and transactions, we monitor all orders, ascertain their status as executed or cancelled, and reconstruct a complete limit order book at each tick, thereby emulating MOEX’s spoofing detection process, yet with more limited data sets. To the best of our knowledge, we are among the first in employing this degree of granularity to tackle the spoofing issue. All of our algorithms are rigorously trained on a dataset comprising suspicious spoofing manipulation cases, flagged by the exchange. Our findings indicate

that machine learning algorithms trained on publicly available data can successfully replace the necessity for proprietary data exclusive to market participants, including trader identities and their corresponding trading styles.

A further concern in the existing literature is the reliance of predicted outcomes on the scope and comprehensiveness of the training data. Obtaining a sufficient quantity of appropriately labelled trading data for supervised learning represents a considerable challenge. Existing research on manipulation detection largely utilizes first and second-level data within the order book, often restricted to the top five tiers of depth, which may not have the sufficient informational depth to detect spoofing. We circumvent this limitation by adopting a more comprehensive analytical perspective, leveraging a dataset that encompasses up to fifty tiers within the order book. This allows us to examine a broader range of potential market state indicators, inclusive of novel metrics such as the order book spread and the order book filling ratio. We categorize our set of indicators into three distinct groups: market quality indicators, trade-related metrics, and operational state descriptors of the order book. Using data-driven feature importance selection, we distill hundreds of variables down to those of significance in forecasting the optimal market conditions for manipulators to introduce orders. This approach allows us to encapsulate key economic dynamics prevalent in limit order markets, yet remains manageable through the implementation of dimensionality reduction techniques.

The majority of contemporary studies concentrate on the application of one or two machine learning algorithms. In contrast, in this paper we present a thorough analysis of the out-of-sample performance of various machine learning approaches, including regularisation methods (lasso, elastic net), tree-based algorithms (Random Forest, XGBoost, and Decision Tree), and Neural Networks. Our objective is to conduct a complete comparison across machine learning methods and to present data regarding the mechanisms by which certain strategies enhance the accuracy of forecasting spoofing manipulation. In the comparison, we calibrate the algorithms using the data the previous five trading days and thereafter perform prediction analysis for the following 10-, 30-, and 60-minute intervals. Through the aggregation of probabilities yielded by each individual machine learning algorithm, we formulate a composite real-time spoofing likelihood score. Our results confirm the superior forecasting performance of our methodology with respect to other machine learning methods, thereby affirming the real-time responsiveness and effectiveness of our RTSP metric in

identifying market states with elevated spoofing propensities. We hope that our approach may be of interest to regulatory bodies, enhancing the detection of spoofing within the limit order book (LOB) while simultaneously augmenting market quality. By incorporating an RTSP-based spoofing risk assessment, market participants can also adjust their trading strategies, thereby altering the market dynamics and rendering conditions less amenable for low-risk manipulator gains, thus inherently improving market quality.

The empirical results show that no individual algorithm consistently outperforms the other. We demonstrate that the RSTP measure, a forecast combination of the three highest-performing models, achieves better out-of-sample predictive performance compared to any single algorithm. Our analysis provides compelling evidence of the model’s suitability for imbalanced datasets, such as the real-time tick data used in our empirical investigation. It also facilitates the identification of intervals featuring potentially suspicious and fraudulent activities within the order book.

The remaining parts of the paper are structured as follows. Section 2 delineates the data and the variables. The subsequent sections of the paper focus on the methodology employed in our investigation (Section 3) and the findings obtained for spoofing prediction (Section 4). In Section 5, we provide the innovative forecasting metric RTSP and demonstrate its predictive efficacy. Conclusive remarks are presented in Section 6.

2. Data collection and processing

In our empirical study, we utilise high-frequency market data spanning six months from January to June 2019, comprising a total of 103,412 observations. Our data is derived from two datasets acquired from the Moscow Exchange (MOEX).

The first dataset contains information on 51,706 cancelled orders of 10 liquid stocks, identified by MOEX, according to their internal algorithm, as potential spoofing orders. This dataset contains details on security ID, timestamp, transaction type (buy/sell), volume, and price. Data providers do not disclose the algorithms employed by the exchange surveillance department for spoofing detection. To the best of our knowledge, they monitor potential spoofing orders by analysing the order’s impact on the market, including its effect on spread tightening, its duration, and various other factors. Furthermore, the exchange monitors the trading behaviour of each trader and can identify possible spoofers. Suspicious spoofing orders in this dataset are discovered using information on verified spoofing cases detected from 2010 to 2019 in the UK, Europe, and the USA.

The second dataset comprises all stock market orders and includes for each order: security code, record number, order type (sell or buy), timestamp in microseconds, order number, price, volume, action type (placed, cancelled, executed), trade number if executed, and trade price if executed. This dataset encompasses all trades, all orders, and the best bid and ask prices. This information enables us to reconstruct the complete limit order book (LOB) to the maximum available level at any given time and for every stock. We create the book with a depth of fifty levels.

We compile our dataset by matching suspicious spoofing orders recognised by MOEX with the second dataset of trades and orders. This enables us to monitor the lifespan of the manipulative order and ascertain its order book price level and cancellation time. We refer to the spoofing orders as "True" orders. Subsequently, we randomly choose non-spoofing orders from the same timeframe and label them as "False" orders. Table 1 presents a description and general statistics of the spoofing orders.

Table 1: Overview and descriptive statistics of the spoofing orders

data format:	stock id, timestamp, buy or sell, price, volume
period:	six months from January to June 2019
list of stocks:	'NVTK', 'ROSN', 'YNDX', 'GMKN', 'FIVE', 'MGNT', 'POLY', 'PLZL', 'TATN', 'SNGS'
quantity:	51 706 spoofing cases
price level:	on average, spoofing orders are placed at the fifth price level above or below the best quotes in the order book
lifetime:	the average duration of a spoofing order is 0.0045 minutes, with a maximum duration of 0.33 minutes.
max price level:	the maximum price level for the spoofing orders is 48
orders frequency:	spoofing cases occur daily for each stock
profitable execution:	84% of spoofing cases do not involve profitable trades from the other side of the book
buy or sell:	45% of spoofing orders are buy orders
distribution of spoofing cases:	orders emerge uniformly throughout the day, with the exception of YNDX, where orders arise exclusively post-4 PM

For the ten stocks, we reconstruct the limit order book at every tick. We focus on price levels determined by the stock's smallest price increment, and we include levels even when they have zero volume. Each reconstructed order book comprises fifty ask and bid price levels. Each price level in the rebuilt order books denotes the number of submitted orders since the start of the trading day, subtracting cancelled and executed orders at that specific price level. The data structure allows us to pinpoint the precise moment the suspicious spoofing order was executed, together with the market conditions immediately preceding that order, which include the recreated historical LOB, trading activity, volume, and price. Our analysis is restricted to the regular trading session, intentionally leaving out the first and last thirty minutes of each day due to the typically high volatility and wider spreads observed during these times (Cartea et al., 2019).

In the first part of our analysis, the dataset is divided into two subsamples. A training and validation sample (in-sample) which includes 75% of the full sample, for a total of and a testing sample which includes the remaining 25% of the sample. The model development process occurs within the in-sample dataset, whereas model validation and assessment are executed on the testing sample. In the second part of our analysis, we conduct a forecasting assessment of the algorithms on the full sample, considering real-time prediction of spoofing at the next tick, based on information from the last five days of trading only.

In developing our model specification, we examine an expanded range of features categorised as market quality variables, trade variables and their frequency, and order book variables. We consider the following measures as indicators of the market quality:

1. Spread measures:

- *QS*. Quoted spread is $(a - b)/m$, where a is the best ask, b is the best bid, and m is the midquote;
- *QS_delta*. The difference in quoted spreads measures ten and twenty seconds before the spoofing order.
- *QS_delta_t2*. The difference in quoted spreads measures ten and thirty seconds before the spoofing order.
- *ES*. Effective (half) spread is $(p - m)/m$, where p is the trade price and m is the prevailing midquote (prior to execution) (Lee, 1993; Blume and Goldstein, 1992).

2. Short-term volatility measures:

- *VOL_1_Nmin*. N-minute price volatility is a standard deviation of the midquotes within the interval divided by the last midquote (Aitken and Frino, 1996). $N \in \{1, 2, 5, 10\}$ minutes.
- *VOL_2_Nmin*. N-minute price volatility is measured as a standard deviation of price returns, where $N \in \{1, 2, 5, 10\}$ minutes.
- *VOL_3_Nmin*. Average realized volatility in the time interval is measured as $1/n \sum_{s=1}^N |\ln(m_{t-s} - m_{t-s-1})|$, where m_s is the midquote at end of minute s , and $N \in \{1, 2, 5, 10\}$ minutes.

When developing trade variables and their frequency, the selection of intervals must correspond with the primary research issue. Given that our data comprises orders with an average lifespan of less than one second, we examine short time frames of under 10 minutes. One-minute intervals align with studies on quote stuffing events (Egginton et al., 2016), whereas 10-minute intervals are employed by Cartea et al. (2019) and Hasbrouck and Saar (2013). To enhance clarity, we use intervals of 1, 2, 5, and 10 minutes and take into account the following:

1. *Hour* denotes a trading period from 10 AM to 6 PM during which the tick associated with the spoofing order occurs.
2. *UF*. The ultra-fast activity measure is the ratio of limit orders that are filed and subsequently cancelled rapidly, relative to the total number of cancelled orders (Cartea et al., 2019). *UF_X_N* denotes an ultra-fast activity metric that is cancelled within X milliseconds, where $X \in \{1, 10, 50, 100, 600\}$ milliseconds within a one-minute interval, and N signifies the number of minutes prior to the occurrence of spoofing order, where $N \in [1; 10]$ minutes. For instance, a spoofing order was executed at 11:40, therefore *UF_10_5* represents a proportion of the limit orders cancelled within 10 milliseconds, measured within the minute from 11:45 to 11:46.

The LOB state represents a "snapshot" of the book whenever an action takes place, such as a limit order submission, cancellation, or execution. MOEX supplies data in the format HH-MMSSZZZXXX, allowing for a minimum time increment of 0.000001 seconds.

Nevertheless, contiguous LOB states may exhibit a temporal disparity of several seconds during periods of moderate activity. We examine the order book condition immediately before to the placement of the spoofing order. The average duration of spoofing orders is 0.005 seconds, and distinct order book variables should be calculated for the ask and bid sides of the limit order book. To determine the order book condition before a manipulative order, it is important to recognize that such orders can be either bids or asks. Could this suggest that the market conditions are particularly attractive to manipulators for placing specific bid or ask orders? We explore this topic by splitting the data into two categories: buy orders and sell orders. Furthermore, we compute the order book depth variables independently for both sides of the limit order book.

1. Order book imbalance. The volume imbalance of the LOB is defined as the difference between the volume at the best bid and the volume at the best ask, divided by the total volume at both

the best bid and ask price levels (Cartea et al., 2020). We calculate the following imbalance measures:

- Total order book imbalance:

$$IMB = (SB - SA)/(SA + SB),$$

where SB represents the total volume of orders at all bid price levels, and SA denotes the total volume of orders at all ask price levels.

- Order book imbalance up to price level i :

$$IMB_i = (SB_i - SA_i)/(SA_i + SB_i),$$

where $i \in \{0, 1, 2, 3, 4, 5\}$ represents a price level in the LOB, with $i = 0$ denoting the best ask and bid levels.

- Order book imbalance prior to the price threshold of the spoofing order:

$$IMB_order = (SB^* - SA^*)/(SA^* + SB^*),$$

where SB^* and SA^* are the cumulative volumes of orders from the bid and ask sides of the order book, respectively, that fall inside the price range specified by the price level with the spoofing order.

- Order book imbalance change:

$$IMB_delta = IBM_10 - IBM_20,$$

where IBM_10 denotes an order book imbalance assessed ten seconds before to the manipulative order placement. IBM_20 represents the order book imbalance twenty seconds before to the manipulative order placement.

$$IMB_delta_t2 = IBM_10 - IBM_30,$$

where IBM_30 represents the variable for the order book thirty seconds before to the manipulative order placement.

2. Order book spread. The bid-ask spread is a traditional indicator of liquidity. It is a trade-based effective cost estimate that has been widely used in studies of market quality. We introduce a measure of order book spread to indicate hidden liquidity inside the LOB that could be seen not as a snapshot of market quality but as potential longer-lasting liquidity.

- Order book spread:

$DistNormN = (AvAskN - AvBidN)/m$, where $AvAskN$ and $AvBidN$ represent the average ask and bid prices, respectively, that correspond to $N \in \{10, 20, 50\}$ ask and bid price levels, including those with zero volume; m denotes a midquote;

- Compressed order book spread:

$DistNormCleanN$ is identical to $DistNormN$, except that it excludes price levels with zero volume. $N \in \{10, 20, 50\}$, excluding zero volume levels.

3. Order book depth. Ranaldo (2004) utilises similar metrics, including pending volume, defined as the number of shares divided by ten thousand on the same or opposite market side as the incoming trader. Biais et al. (1995) demonstrate that thin books generate orders, whereas thick books lead to trades. We apply a similar approach by introducing the subsequent measures:

- Mean order book depth:

$DistVolN = (VolAsk + VolBidN)/Vol_m$ represents the average cumulative volume in the limit order book within $N \in \{10, 20, 50\}$ basis points of the best bid and ask (Cartea et al., 2019). $VolAskN$ and $VolBidN$ represent the cumulative volumes on the ask and bid sides of the order book, respectively, within N price level.

$Vol_m = (VolAsk + VolBid)/2$ represents the mean volume at the best bid and ask price levels. This metric considers the tick size by evaluating the depth at specified intervals in relation to the current best bid and ask. We standardise this metric using the average volumes at the best bid and ask.

- Mean compressed order book depth:

$DistVolCleanN$ is equivalent to $DistVolN$, excluding price levels with no volume. $N \in \{10, 20, 50\}$, excluding zero volume levels. For instance, $N = 20$, although we detect two price levels on the ask side with no volume. We must calculate the cumulative volume up to the twenty second price increment from the best ask.

4. Order book filling ratio. The metric is a ratio of non-zero volume price levels in the LOB to the total quantity of levels. The order book displays multiple observable pricing levels; nevertheless, the disparity between next levels may exceed the minimum price increment. The order book may contain orders at each price level or have numerous vacant levels. We

suggest the introduction of the following innovative variables to quantify this ratio:

- $FRA = FA/TotalA$ and $FRB = FB/TotalB$, where FA and FB denote the quantities of price levels in the order book containing at least one ask or bid limit order, respectively. $TotalA$ and $TotalB$ represent the highest price levels from the order book’s ask and bid sides, respectively.

- $FRA_delta = FRA_10 - FRA_20$ and $FRB_delta = FRB_10 - FRB_20$,

where FRA_10 and FRB_10 represent the order book filling ratios assessed ten seconds before to the spoofing order placement. FRA_20 and FRB_20 represent the order book filling ratios twenty seconds before to the manipulative order placement.

$$FRA_delta_t2 = FRA_10 - FRA_30 \text{ and } FRB_delta_t2 = FRB_10 - FRB_30,$$

where FRA_30 and FRB_30 are the order book filling ratios for the order book thirty seconds prior to the spoofing order placement.

The variable generating procedure results in an extensive array of predictors as prospective candidates for our modelling procedures. Our choice to examine a broad range of predictors is driven by the inconclusive evidence in the existing research regarding which variables should be incorporated in predicting spoofing manipulation (Cao et al., 2015; Tao et al., 2022). Our decision is motivated by the desire to avoid overlooking any critical information.

Initially, we reduce the dimensionality of the suggested variables that represent the current market conditions, allowing the core models to be constructed on a subset of variables that include the most pertinent ones for the purpose of the study. As a first step, to ensure a consistent framework for comparison across different stocks and to facilitate interpretability, each variable is standardized by eliminating outliers, defined as deviations beyond three standard deviations from the mean of each feature. Subsequently, given the multitude of explanatory factors, we exclude those that predominantly provide redundant information in predicting the dependent variable by regularisation procedures. We employ LASSO (Least Absolute Shrinkage and Selection Operator) as a regularisation method for variable selection, as outlined by (Tibshirani, 1996; Hastie et al., 2009) (see Appendix 7.1) to assess simultaneously the predictive power of features. However, while

LASSO is capable of accommodating pre-specified non-linearities and interactions, it may exhibit sub-optimal performance in the presence of highly correlated predictors (Zou and Hastie, 2005). The sample correlation matrix of our candidate predictors is provided in Appendix 7.2. Due of the significant connection among predictors, consistent with the results of Zou and Hastie (2005), we utilise the Elastic Net to strengthen our findings from LASSO. We choose a parameterization with an alpha of 50% (Appendix 7.1)). The regressors incorporated into our model are chosen for their ability to reduce the Root Mean Square Error (RMSE). As the outcome of the variable selection process, we retain 67 regressors as predictors of spoofing manipulation trading activity. The full list of predictors is reported in documented in Appendix 7.3. At this juncture, it is imperative to ascertain that the ultimate system of variables stays resilient across various model specifications. Consequently, we broaden our research by implementing a Stepwise Logit (SL) model definition. Comprehensive results can be obtained upon request from the authors. The chosen variables are resilient to the model specification, indicating that our primary results about the predictive efficacy of each model are not adversely affected by the variable selection technique employed.

3. Model Development

To forecast the likelihood of spoofing manipulation at the subsequent tick, we evaluate various machine learning techniques, including deep learning approaches, and conduct an extensive comparison. Specifically, our analysis encompasses the following classical machine learning models: Logistic Regression, k-Nearest Neighborhoods, Support Vector Machines, Stochastic Gradient Descent, Naive Bayes, Random Forest, Decision Trees, XGBoost, Gradient Boosting, and Adaptive Markov Chain. For the Neural Networks, we include Perceptron, LSTM and GRU. All these models are thoroughly described in the literature.

The *Logistic Regression* is used as a benchmark. It is a non-linear fully parametric approach and has several advantages: it is easy to fit and to interpret. Statistical significance tests can be conducted with ease. Nevertheless, parametric methods possess a drawback: they inherently rely on stringent assumptions, and if the designated functional form deviates much from reality, the accuracy of the parametric method will be suboptimal.

The most straightforward non-parametric method we examine is the *k-Nearest Neighbors (kNN)*. kNN classifies objects according to the categories of their closest neighbours in the dataset. Distance measures are employed to identify the nearest neighbour. The kNN technique identifies the k nearest

training cases inside the feature space and use their average prediction. kNN predictions posit that proximate items exhibit similarity.

Support Vector Machines (SVM) transform inputs into elevated-dimensional feature spaces. The SVM classifies data by identifying the linear decision boundary, or hyperplane, that distinguishes data points of one class from those of another class. This machine-learning approach delineates the attribute space using a hyperplane, optimising the margin between occurrences of disparate classes or class values. It is applicable when the researcher requires a classifier that is straightforward, easily interpretable, and precise. A considerable body of research highlights the efficacy of Support Vector Machines (SVMs) in detecting market manipulation (Ögüt et al., 2009; Khodabandehlou and Golpayegani, 2022), as they mitigate the risk of overfitting and lessen the necessity for laborious cross-validation in hyperparameter selection. Their primary constraint is the absence of interpretability.

Decision trees have resilience to varied scaling and robustness against outliers, and have been employed by numerous authors as techniques for predicting spoofing manipulation. For example, when trading volume above the 75th percentile, a split occurs depending on whether the quoted spread is above or below the median; otherwise, the split is determined by the 15th percentile of order book imbalance. Each resultant leaf is subsequently allocated an anticipated frequency of spoofing action, consistent with historical averages. In recent years, ensemble algorithms, especially tree-based algorithms, have become essential tools for tackling prediction and classification problems across various fields, resulting in significant successes (Zhou, 2012). Instead of depending on a single model, tree-based methods combine multiple individual tree models to achieve optimal prediction performance. The *Random Forest*, as presented by Ho (1995), functions as a supervised ensemble learning method utilising decision trees. It consolidates outcomes from numerous random decision trees (Breiman, 2001), averaging the results of various trees, each constructed from a bootstrapped sample derived from a random selection of all predictors in the original dataset.

XGBoost, which proficiently executes Gradient Tree Boosting, is detailed in Chapter 10 of Hastie et al. (2017). Random Forest utilises averaging over random trees, a method known as bagging, whereas XGBoost emphasises constructing each new tree to address instances that prior trees struggled with, a technique referred to as boosting. Generally, boosting yields more accurate forecasts than bagging, albeit the estimation process is considerably slower. XGBoost significantly

mitigates this deceleration, rendering Gradient Tree Boosting almost as rapid as Random Forest. Furthermore, it acknowledges the tendency of trees to overfit and enforces penalties on trees with an excessive number of leaves, so promoting simpler and more succinct trees—a process known as regularisation. As discussed in Chen et al. (2015), the XGBoost algorithm is equally adept at providing computational efficiency, particularly when dealing with sizable datasets. Moreover, XGBoost, akin to Random Forest, is adept at leveraging hyperparameters which aligns well with our data requirements, specifically our need to wield various order book variables for spoofing prediction, incorporating clusters, rates, and other parameters within a unified algorithm.

4. Model Validation

To evaluate model performance, we run a comprehensive validation exercise regarding predictive capability throughout the testing period. To ensure robustness of our findings to the validation procedure, we consider several different cross-validation approaches in controlling for overfitting.

4.1. Validation measures

To differentiate among competing forecasts, we evaluate the predictions utilising a set of metrics, specifically *accuracy*, *precision*, *recall*, and *f1 – score*. These measurements address any potential misinterpretations of model performance that may occur when utilising conventional performance metrics reliant on total accuracy. We use *accuracy*, *precision*, *recall* and the *f1 – score*, defined as follows:

$$Accuracy = \frac{tp + tn}{tp + tn + fp + fn}, \quad Precision = \frac{tp}{tp + fp},$$

$$Recall = \frac{tp}{tp + fn}, \quad F1 - score = \frac{2 * Recall * Precision}{Recall + Precision},$$

where, *tp* denotes the quantity of true positive values, *tn* signifies the quantity of true negative values, *fp* represents the quantity of false positive values, and *fn* indicates the quantity of false negative values.

The *accuracy* metric is the ratio of accurately predicted market states to the total number of predicted market states. It quantifies the proportion of all test samples accurately classified.

The *precision* quantifies the ratio of accurately identified samples within a population that are classed as positive labels. Consequently, it addresses the inquiry: Among all "True" projected market situations featuring spoofing orders, how many genuinely contained spoofing orders in the

limit order book. For instance, a *precision* of 0.72 for "True" spoofing orders indicates that 72% of all anticipated market situations featuring spoofing orders in the limit order book were accurately recognised by the algorithm. In a similar vein, a *precision* of 0.64 for "False" spoofing orders indicates that 64% of anticipated market conditions devoid of spoofing orders in the limit order book were accurately recognised.

The *recall* or sensitivity, quantifies a classifier's capacity to accurately recognise positive labels. The *recall* addresses the question: Among all genuine "True" market conditions featuring spoofing orders, how many were accurately anticipated by the algorithm? For instance, a *recall* of 0.75 for "True" spoofing orders indicates that 75% of market conditions featuring spoofing orders in the limit order book were accurately recognised by the machine learning algorithm. Likewise, a recall of 0.79 for "False" spoofing orders indicates that 79% of market conditions without of spoofing orders were accurately detected.

The *f1 – score* combines *precision* and *recall* into a single metric as their harmonic mean. By definition, it is never greater than the geometric mean and approaches the smallest value, so diminishing the influence of significant outliers while amplifying the effect of minor ones. The *f1 – score*, consequently, tends to favour balanced systems. To test for equal predictive ability of the models we use the Diebold Mariano (DM) test on model pairs for each of the accuracy measures. Finally, we rely on the Model Confidence Set test to rank models forecasting performances.

4.2. Validation Results

The fully balanced nature of our original sample may facilitate the training of the models. To investigate the robustness of our approach to the more realistic scenario of an unbalanced data set, we analyze model performance for the case of 1:2 True-False proportion. Consequently, we generate a new training sample, including an additional 51,706 randomly selected "False" orders. We assess model performance in the "balanced" (original) and "unbalanced" dataset. As already mentioned, the validation period refers to 25% of the full data. Our analysis reveals an average prediction accuracy ranging from 50% and 76% (Table 2) for the balanced sample and indicates the Random Forest as the model with best performance. These findings are also corroborated by the Diebold-Mariano test, which enables us to reject the null hypothesis of equal prediction performance among all models compared to the Random Forest. In addition, the DM test indicates that the Decision Tree and XGBoost models either outperform or exhibit comparable performance to the

other models (see Appendix 8.4). Results from the Model Confidence Set test confirm the Random Forest model as the optimal choice in term of prediction accuracy. Subsequently, in pursuit of selecting the second-best model, we dismiss the Random Forest from consideration, leading us to identify two other models, Decision Trees and XGBoost that exhibit similar and commendable performance levels (refer to Appendix 8.5). Following this first step of analysis, the Random Forest, Decision Tree, and XGBoost undergo calibration to achieve optimization of the hyperparameters (refer to Appendix 7.6). The table below presents the outcomes of frequently utilised machine learning models with a "True" to "False" ratio of 1:1, partitioning the data into 75% training and 25% validation sets.

Table 2: Accuracy of predictions made by machine learning models. The table presents the outcomes of frequently utilised machine learning models with a "True" to "False" ratio of 1:1, dividing the data into 75% training and 25% validation sets.

Logistic regression:	52%
Support Vector Machines (SVM):	57%
K-nearest neighbors (KNN):	68%
Naive Bayes (NB):	50%
Stochastic Gradient Descent (SGD):	52%
Decision Tree (DT):	68%
Random Forest (RF):	76%
Gradient Boosting (GB) :	67%
XGBoost:	68%
Adaptive Markov Chains:	50%
LSTM Neural Network (10 hidden layers):	55%
LSTM Neural Network (5 hidden layers):	55%
GRU neural network:	51%
Perceptron neural network:	53%

In the initial phase of the investigation, three algorithms — Random Forest, Decision Tree, and XGBoost — accurately anticipate between 66% and 78% of spoofing cases for both balanced and unbalanced datasets.

4.3. Robustness

We examine the robustness of the results across different cross-validation settings to mitigate overfitting. Cross-validation is a resampling technique employed to assess the reliability of predictive performance of machine learning models on a constrained data sample. The technique involves a singular parameter K , which denotes the number of groups into which a specific data sample will be divided.

We commence with k-fold cross-validation for each machine learning model to derive the average prediction accuracy. It typically yields a more accurate or less optimistic assessment of the model’s performance compared to alternative methods, such as a basic train-test split. The procedure is as follows: first, the dataset is randomly shuffled; second, it is divided into K groups, using each group as a hold-out or test dataset while fitting a model on the training set and evaluating it on the test set; finally, the evaluation score is retained and the model is discarded. Subsequently, we encapsulate the model’s efficiency utilising the sample of assessment scores. Afterwards, we evaluate the model’s predicting effectiveness by calculating the mean accuracy of the predictions for each fold. We employ K-fold, Stratified K-fold, and Shuffle cross-validations, separating the spoofing events into K non-overlapping segments in chronological order. We do cross-validation using several values of K (3, 4, 5, 6, 8, 10) to assess the stability of the results.

In k-fold cross-validation, each fold serves as a validation set while the remaining folds function as the training set. This validation technique is considered inappropriate for imbalanced datasets, as the model will not receive sufficient training due to the inadequate representation of each class. Thus, for imbalanced data, we employ the shuffle and block approach to train the models.

Shuffling the dataset guarantees that the model encounters a varied sequence of samples in each segment, hence mitigating the risk of memorising the training data’s order and overfitting to particular patterns. The blocked k-fold cross-validation method resembles the aforementioned standard process. The distinction lies in the absence of initial randomisation of observations.

Stratified K-fold cross-validation, an improved variant of k-fold cross-validation, divides the dataset into K equal folds. Each fold maintains an identical proportion of instances of the desired variables. This approach facilitates the handling of skewed datasets. Finally, we employ the shuffle cross-validation strategy, which entails partitioning the entire dataset according to a specified percentage while maintaining a distinct train-test split ratio. This method enables us to obtain the test errors and evaluate the results.

4.4. Forecasting Results for balanced and imbalanced datasets

In the context of a balanced dataset, the three machine learning algorithms used in the initial phase of the investigation demonstrate predictive accuracy ranging from 66% to 78%. Tables 3, 4, 5, 6 below show the results.

Table 3: Accuracy of machine learning models for balanced datasets. The table presents the outcomes of the Random Forest, XGBoost, and Decision Tree machine learning models, with an equal proportion of "True" and "False" spoofing orders set at 1:1, and the data divided into 75% training and 25% validation sets. *Accuracy* quantifies the proportion of testing samples that are classified properly. *Recall* quantifies a classifier's capacity to accurately detect positive labels. *Precision* quantifies the ratio of accurately recognised samples within a population that are classed as positive labels. *F1 – score* integrates *precision* and *recall* into a singular metric, defined as $F1 - score = (2 * Recall * Precision) / (Recall + Precision)$.

	Random Forest			XGBoost			Decision Tree		
	Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score
False	0.74	0.84	0.79	0.78	0.63	0.69	0.64	0.72	0.68
True	0.82	0.71	0.76	0.70	0.82	0.75	0.69	0.60	0.64
Accuracy		0.78			0.73			0.66	

Different cross-validation approaches illustrate that the models performance is robust to the validation approach. It is noteworthy that K-fold cross-validation with blocks indicates that XGBoost surpasses the Random Forest algorithm, whereas shuffle K-fold cross-validation implies the superiority of Random Forest. There is no uniquely dominant algorithm across different cross validation approaches. This suggest that combining different algorithms may improve the overall forecasting performance (Claeskens et al., 2016). In the case of imbalanced data, the Random Forest model achieves 82.3% accuracy. Due to the imbalanced dataset, only 57% of "True" orders are accurately predicted, however the prediction accuracy for "False" orders is 95%. The XGBoost model achieves an overall prediction accuracy of 78.2%, with 59% accuracy in predicting "True" spoofing orders and 88% accuracy in predicting "False" spoofing orders. The decision tree exhibits comparable outcomes with slightly less predicting accuracy (Tables 7, 8, 9, 10).

Table 4: K-fold cross-validation assessment of machine learning models using balanced data with a 1:1 True-False ratio. The table presents the outcomes of two cross-validation methods: 1. *Shuffle*: the entire dataset is randomly divided into k parts; 2. *Blocks*: the dataset is organised chronologically into blocks without randomisation.

1. Shuffle				
K	Accuracy (%)	Random Forest	XGBoost	Decision Tree
3	Min accuracy	76.5	72.5	66.0
	Max accuracy	76.9	72.7	66.6
	Average accuracy	76.7	72.6	66.3
4	Min accuracy	77.1	72.4	65.8
	Max accuracy	77.4	72.9	66.4
	Average accuracy	77.2	72.7	66.1
5	Min accuracy	77.2	72.4	65.6
	Max accuracy	77.7	73.3	66.4
	Average accuracy	77.5	72.9	66.1
6	Min accuracy	77.3	72.0	65.7
	Max accuracy	78.2	73.4	66.4
	Average accuracy	77.7	72.9	66.1
8	Min accuracy	77.4	72.0	65.9
	Max accuracy	78.3	73.2	66.8
	Average accuracy	77.9	72.7	66.1
10	Min accuracy	77.4	72.4	65.5
	Max accuracy	78.4	73.3	66.9
	Average accuracy	78.0	73.0	66.3
2. Blocks				
K	Accuracy (%)	Random Forest	XGBoost	Decision Tree
3	Min accuracy	56.6	62.0	62.8
	Max accuracy	61.0	66.4	66.7
	Average accuracy	58.4	64.3	64.7
4	Min accuracy	54.9	61.0	59.0
	Max accuracy	61.9	67.9	65.8
	Average accuracy	57.9	64.7	62.2
5	Min accuracy	55.9	63.0	60.1
	Max accuracy	64.2	70.2	69.2
	Average accuracy	59.3	65.9	64.4
6	Min accuracy	55.3	60.0	59.3
	Max accuracy	63.4	68.5	68.4
	Average accuracy	58.6	64.8	64.2
8	Min accuracy	55.3	56.5	56.1
	Max accuracy	68.7	68.6	69.1
	Average accuracy	58.9	64.5	62.8
10	Min accuracy	54.7	59.6	57.2
	Max accuracy	70.3	71.9	71.3
	Average accuracy	59.8	65.9	64.1

Table 5: Stratified K-fold cross-validation assessment for machine learning models utilising balanced data with a 1:1 True-False ratio.

K	Accuracy (%)	Random Forest	XGBoost	Decision Tree
3	Min accuracy	76.4	72.0	65.7
	Max accuracy	76.9	73.0	66.3
	Average accuracy	76.6	72.5	66.1
4	Min accuracy	77.0	72.2	65.3
	Max accuracy	77.3	72.9	66.8
	Average accuracy	77.1	72.7	66.0
5	Min accuracy	77.2	71.9	65.8
	Max accuracy	78.0	73.4	66.8
	Average accuracy	77.5	72.8	66.2
6	Min accuracy	77.4	72.5	65.8
	Max accuracy	78.0	73.5	66.2
	Average accuracy	77.6	72.8	66.0
8	Min accuracy	77.5	72.4	65.5
	Max accuracy	78.4	73.5	67.2
	Average accuracy	77.9	73.0	66.2
10	Min accuracy	77.5	71.7	65.1
	Max accuracy	79.2	73.9	67.1
	Average accuracy	78.0	72.9	66.2

Table 6: Shuffle K-fold cross-validation for machine learning models utilising balanced data with a 1:1 True-False ratio, training on 60% of the dataset and verifying on 20% of the dataset.

K	Accuracy (%)	Random Forest	XGBoost	Decision Tree
3	Min accuracy	76.0	71.9	65.8
	Max accuracy	76.3	72.7	66.1
	Average accuracy	76.2	72.4	65.9
4	Min accuracy	76.1	72.2	65.5
	Max accuracy	76.6	72.6	66.8
	Average accuracy	76.3	72.4	65.9
5	Min accuracy	75.8	72.1	65.8
	Max accuracy	76.7	73.0	66.4
	Average accuracy	76.2	72.6	66.2
6	Min accuracy	75.6	71.7	65.8
	Max accuracy	76.6	72.7	66.4
	Average accuracy	76.2	72.3	66.1
8	Min accuracy	75.7	71.8	65.7
	Max accuracy	76.7	72.8	66.4
	Average accuracy	76.3	72.2	66.1
10	Min accuracy	75.9	71.6	65.6
	Max accuracy	76.6	72.9	66.6
	Average accuracy	76.4	72.3	66.1

Table 7: Accuracy of machine learning models for unbalanced datasets. This table presents the outcomes of the Random Forest, XGBoost, and Decision Tree machine learning models, where the proportions of "True" and "False" orders are set at 1:2, with the data divided into 75% training and 25% validation sets. *Accuracy* quantifies the proportion of testing samples that are classified properly. *Recall* quantifies a classifier's capacity to accurately detect positive labels. *Precision* quantifies the ratio of accurately recognised samples within a population that are classed as positive labels. *F1 - score* integrates *precision* and *recall* into a singular metric, defined as $F1 - score = (2 * Recall * Precision) / (Recall + Precision)$.

	Random Forest			XGBoost			Decision Tree		
	Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score
False	0.81	0.95	0.88	0.81	0.88	0.84	0.76	0.85	0.80
True	0.85	0.57	0.69	0.71	0.59	0.65	0.61	0.46	0.53
Accuracy		0.82			0.78			0.72	

Table 8: Stratified K-fold cross-validation assessment for machine learning models using unbalanced data with a "True" to "False" ratio of 1:2.

K	Accuracy (%)	Random Forest	XGBoost	Decision Tree
3	Min accuracy	81.6	77.9	71.3
	Max accuracy	81.9	78.0	72.0
	Average accuracy	81.7	77.9	71.6
4	Min accuracy	81.8	77.9	71.7
	Max accuracy	82.2	78.4	72.1
	Average accuracy	82.1	78.1	71.9
5	Min accuracy	81.9	77.8	71.5
	Max accuracy	82.5	78.4	72.3
	Average accuracy	82.3	78.2	71.9
6	Min accuracy	82.0	77.6	71.2
	Max accuracy	82.7	78.4	72.2
	Average accuracy	82.3	78.2	71.8
8	Min accuracy	82.1	77.8	71.3
	Max accuracy	83.0	78.5	72.2
	Average accuracy	82.6	78.3	71.8
10	Min accuracy	82.1	77.8	71.2
	Max accuracy	83.1	78.6	72.3
	Average accuracy	82.7	78.3	71.8

Table 9: K-fold cross-validation assessment of machine learning models using imbalanced data with a 1:2 True-False ratio. The table presents the outcomes of two cross-validation methods: 1. *Shuffle*: the entire dataset is randomly divided into k parts; 2. *Blocks*: the dataset is organised chronologically into blocks without randomisation.

1. Shuffle				
K	Accuracy (%)	Random Forest	XGBoost	Decision Tree
3	Min accuracy	81.7	77.8	71.8
	Max accuracy	81.8	77.9	72.0
	Average accuracy	81.7	77.9	71.9
4	Min accuracy	81.9	77.9	71.7
	Max accuracy	82.3	78.2	72.2
	Average accuracy	82.0	78.0	71.9
5	Min accuracy	82.0	77.8	71.6
	Max accuracy	82.5	78.3	72.1
	Average accuracy	82.2	78.1	71.8
6	Min accuracy	82.2	77.9	71.7
	Max accuracy	82.7	78.4	72.1
	Average accuracy	82.5	78.2	71.9
8	Min accuracy	82.2	78.1	71.3
	Max accuracy	83.0	78.7	72.3
	Average accuracy	82.6	78.3	71.9
10	Min accuracy	82.3	78.0	71.4
	Max accuracy	83.2	78.6	72.1
	Average accuracy	82.7	78.3	71.8
2. Blocks				
K	Accuracy (%)	Random Forest	XGBoost	Decision Tree
3	Min accuracy	66.4	68.9	69.3
	Max accuracy	66.5	70.3	70.7
	Average accuracy	66.4	69.7	69.8
4	Min accuracy	66.1	67.6	67.5
	Max accuracy	66.7	72.1	71.6
	Average accuracy	66.5	69.8	69.6
5	Min accuracy	64.7	69.2	68.0
	Max accuracy	70.6	72.7	71.3
	Average accuracy	67.3	70.8	69.7
6	Min accuracy	65.8	68.1	66.2
	Max accuracy	66.9	71.6	71.7
	Average accuracy	66.5	69.7	69.6
8	Min accuracy	64.0	66.0	66.2
	Max accuracy	68.3	73.4	73.7
	Average accuracy	66.6	69.7	69.6
10	Min accuracy	65.0	66.9	66.2
	Max accuracy	74.8	77.3	74.5
	Average accuracy	67.4	71.0	69.8

Table 10: Shuffle K-fold cross-validation for machine learning models utilising imbalanced data with a 1:2 True-False ratio, training on 60% of the dataset and verifying on 20% of the dataset.

K	Accuracy (%)	Random Forest	XGBoost	Decision Tree
3	Min accuracy	81.0	77.8	71.6
	Max accuracy	81.3	78.0	71.9
	Average accuracy	81.2	77.9	71.8
4	Min accuracy	81.1	77.7	71.3
	Max accuracy	81.6	78.0	72.1
	Average accuracy	81.4	77.9	71.6
5	Min accuracy	81.2	77.7	71.7
	Max accuracy	81.5	78.1	72.2
	Average accuracy	81.4	78.0	71.9
6	Min accuracy	81.2	76.9	71.3
	Max accuracy	81.6	78.2	72.2
	Average accuracy	81.4	77.6	71.8
8	Min accuracy	81.0	77.3	71.4
	Max accuracy	81.7	78.0	72.1
	Average accuracy	81.3	77.7	71.7
10	Min accuracy	80.9	77.4	71.3
	Max accuracy	81.9	78.4	72.3
	Average accuracy	81.5	77.8	71.8

To better understand the performance of the models on the imbalanced dataset, we execute the selected algorithms while augmenting the fraction of "False" orders to 15 by reducing the number of "True" spoofing orders. The results, displayed in Table 11, affirm that the validity of the method is significantly dependent on the proportion of the training data.

Recent years have seen the proposal of numerous advanced learning strategies to tackle classification issues in imbalanced datasets, encompassing both algorithmic and data-centric approaches. The algorithm-driven strategy, or classifier-level approach, maintains a constant training dataset while modifying the inference algorithm to enhance learning pertinent to the minority class. To demonstrate this methodology, we implement the 'BalanceRandomForestClassifier' for imbalanced datasets (Appendix 7.6), which randomly under-samples each bootstrap sample to achieve balance. We achieve a predictive accuracy of 80% with a "True" order prediction rate of 71%. To balance order weights in XGBoost, we substantially altered the parameter 'scale_pos_weight' from 1 to 500 (Appendix 7.6), hence prioritising "True" order predictions by a factor of 500, resulting in an overall accuracy of 81%. Results are shown in Table 12.

Table 11: Accuracy of machine learning methods conditional on data imbalance. This table presents the outcomes of training machine learning algorithms on imbalanced data, obtained by reducing the initial dataset of "True" spoofing orders to attain the proportions indicated in the first column. We divided the data into 75% for training and 25% for validation. We present the measures of *Accuracy* and *Recall*. *Accuracy* quantifies the proportion of testing samples that are classified properly. *Recall* quantifies a classifier’s capacity to accurately detect positive labels. In other terms, *Recall* can be perceived as an accuracy measure for "True" and "False" classifications independently.

True:False	Random Forest			XGBoost			Decision Tree		
	Accuracy	Recall False	Recall True	Accuracy	Recall False	Recall True	Accuracy	Recall False	Recall True
1:1	0.77	0.84	0.70	0.72	0.63	0.82	0.66	0.72	0.59
1:2	0.80	0.95	0.50	0.77	0.88	0.57	0.72	0.90	0.38
1:3	0.83	0.97	0.41	0.81	0.95	0.42	0.77	0.98	0.14
1:4	0.86	0.98	0.36	0.84	0.97	0.35	0.81	0.99	0.13
1:5	0.88	0.99	0.33	0.87	0.99	0.29	0.84	0.99	0.09
1:6	0.89	0.99	0.33	0.89	0.99	0.27	0.86	0.99	0.11
1:7	0.90	0.99	0.28	0.90	0.99	0.23	0.88	0.99	0.09
1:8	0.91	0.99	0.28	0.91	1.00	0.23	0.89	1.00	0.05
1:9	0.92	0.99	0.27	0.92	1.00	0.22	0.90	1.00	0.05
1:10	0.92	0.99	0.24	0.92	1.00	0.20	0.91	1.00	0.06
1:11	0.93	0.99	0.25	0.93	1.00	0.21	0.92	1.00	0.04
1:12	0.93	0.99	0.22	0.93	1.00	0.19	0.92	1.00	0.6
1:13	0.94	0.99	0.24	0.94	1.00	0.21	0.93	1.00	0.3
1:14	0.94	0.99	0.23	0.94	1.00	0.18	0.93	1.00	0.3
1:15	0.95	1.00	0.21	0.94	1.00	0.17	0.94	1.00	0.6

Table 12: Accuracy of machine learning models with adjusted parameters. This table presents the outcomes of adjusted parameters for the Random Forest model utilising the 'BalanceRandomForestClassifier' (1), and the XGBoost machine learning model with modifications of 500 and 10 in the 'scale_pos_weight' parameter (2) and (3), respectively. We employ an imbalanced ratio of "True" to "False" orders at 1:2, dividing the data into 75% for training and 25% for validation. *Accuracy* quantifies the proportion of testing samples that are classified properly. *Recall* quantifies a classifier’s capacity to accurately detect positive labels. *Precision* quantifies the ratio of accurately recognised samples within a population that are classed as positive labels. *F1 – score* integrates *precision* and *recall* into a singular metric, defined as $F1 - score = (2 * Recall * Precision) / (Recall + Precision)$.

	Balanced RF (1)			XGBoost (500) (2)			XGBoost (10) (3)		
	Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score
False	0.85	0.86	0.86	0.83	0.91	0.87	0.79	0.86	0.82
True	0.72	0.70	0.71	0.77	0.63	0.70	0.66	0.55	0.60
Accuracy	0.81			0.81			0.75		

5. A real-time spoofing probability measure

The results from the preceding sections indicate that XGBoost and Random Forest are the most effective models in terms of *precision* and *recall*, respectively. Furthermore cross-validation robustness checks show these two algorithms emerge as the dominant ones in different cross validation approaches.

This section introduces a new forecasting metric, the Real-Time Spoofing Probability (RTSP), derived from the combination of the XGBoost and Random Forest methods. The RTSP seeks to

offer a real-time assessment of the risk associated with engaging with a spoofer. In contrast to liquidity indicators like effective or quoted bid-ask spreads, identifying a spoofing manipulation metric is challenging. Manipulators typically conceal themselves within the order flow of retail trades to evade detection. Empirical measures are generally driven by theoretical analyses of spoofing trade to address these constraints. For instance, Cartea et al. (2020) analyse the trading strategy of an investor who engages in spoofing the limit order book (LOB) and calculates the LOB volume imbalance as a critical metric for the profitability of the spoofing approach. We advocate for a data-driven methodology to assess trading risk in scenarios when spoofing manipulation is highly probable.

To evaluate the forecasting accuracy of the RSTP measure on out-of-sample data, we train models using five consecutive trading days and the current trading day prior to 14:00. Given that the duration of spoofing orders is under one minute, as seen in Table 1, and our model relies on rapidly changing limit order book data, it is imperative to often augment the training set with new information. So, we predict spoofing orders for the upcoming 10, 30, and 60 minutes from 14:00 to 18:00. We forecast each tick on the market condition. We maintain forecasts of 10, 30, and 60 minutes to ensure the validity of our findings. For instance, in a 10-minute setting, we train data from the preceding five trading days and the current morning to predict spoofing risk for each tick, thereafter retraining the model with updated data every 10 minutes.

The model's outputs range from zero to one, indicating the chance of the manipulative spoofing order occurring in the subsequent tick. The RTSP increases when the likelihood of spoofing trade rises. Rounding is not employed in the calculation of the RTSP measure. We train models using a balanced dataset with a 1:1 ratio of "True" to "False" spoofing orders. We then anticipate the imbalanced dataset with a 1:2 ratio of "True" to "False" orders, as this closely resembles the actual trading environment.

We employ two validation methods that involve augmenting the dataset with additional data for training. Figure 3 illustrates the rationale behind the expanding validation methodology as we incorporate additional data for the training phase, while maintaining the testing phase at 10, 30, or 60 minutes. Additionally, we employ a rolling validation method that maintains a consistent training period throughout all splits (Figure 4). The outcomes of both methodologies are presented in Table 13. We predict a total of 70,347 orders, comprising 19,971 "True" orders and 50,376

"False" suspect spoofing orders. Table 13 presents the forecasting outcomes of the three most effective models and RTSP metrics, utilising the two validation methods depicted in Figure 3 and Figure 4.

Table 13: Forecasting performance of RTSP. The table presents the outcomes of machine learning models and the RTSP metric under 10-, 30-, and 60-minute forecasting re-estimation configurations. The column labelled *All correct* indicates the percentage of accurately predicted market states, while the columns *True correct* and *False correct* reflect the percentages of correctly predicted market states with "True" and "False" spoofing orders, respectively. The first figure presents the outcome derived from the expanding validation methodology (Figure 3), wherein additional data is incorporated into the training dataset while maintaining a consistent testing duration of 10, 30, or 60 minutes. The second figure following the slash indicates the outcomes derived from the rolling validation method depicted in Figure 4.

10-minute re-estimation frequency			
Model	All correct (%)	True correct (%)	False correct (%)
XGBoost	67.2 / 66.4	74.4 / 46.9	75.0 / 75.3
Random Forest	68.7 / 68.8	22.2 / 22.1	87.1 / 87.3
Decision Tree	65.0 / 64.5	49.5 / 47.4	71.1 / 71.3
RTSP	68.2 / 68.3	42.5 / 41.9	78.4 / 78.8
30-minute re-estimation frequency			
Model	All correct (%)	True correct (%)	False correct (%)
XGBoost	66.8 / 66.7	47.1 / 46.1	74.6 / 74.8
Random Forest	68.2 / 68.5	21.4 / 21.9	86.8 / 86.9
Decision Tree	65.3 / 64.6	50.2 / 48.7	71.3 / 70.9
RTSP	68.1 / 68.3	42.7 / 42.4	78.1 / 78.6
60-minute re-estimation frequency			
Model	All correct (%)	True correct (%)	False correct (%)
XGBoost	66.3 / 66.4	47.0 / 46.2	74.0 / 74.4
Random Forest	67.9 / 68.2	21.3 / 21.9	86.4 / 86.5
Decision Tree	65.0 / 64.3	50.1 / 48.2	70.9 / 70.7
RTSP	67.9 / 67.8	42.7 / 42.1	77.9 / 78.0

In general, we do not observe substantial variations in predicted accuracy among 10-, 30-, and 60-minute rolling forward predictions. The frequency of re-estimating our model, whether every 10 minutes or every hour, does not much enhance the outcomes. XGBoost and Decision Tree models accurately forecast around 50% of "True" cases and 70-75% of "False" spoofing cases, yielding a total predictive accuracy of about 67%. The Random Forest model accurately predicts only 20-25% of "True" spoofing cases, while achieving an 87% accuracy for "False" spoofing cases, resulting in an overall correct prediction rate of approximately 68% for all events. The RTSP metric outperforms all three algorithms. Various forecasting validation methods yield comparable outcomes.

Furthermore, we evaluate our approach on buy and sell suspect spoofing orders independently. ?? demonstrates that the predicting accuracy for market conditions characterised by buy spoofing orders is slightly better. This example illustrates that models can be trained differently based on the specific goal; for instance, the exchange seeks to assess the risk of spoofing just from the bidding side of the limit order book.

Table 14: Forecasting performance of RTS for buy and sell orders. The table presents the accuracy of machine learning models and the RTSP metric for buy and sell spoofing risk over 10-, 30-, and 60-minute forecasting re-estimation constraints. The column *All Buy* indicates the percentage of accurately anticipated market states involving buy spoofing orders, whereas the columns *True Buy* and *False Buy* represent the percentage of accurately forecasted market states for "True" and "False" buy orders, respectively. The identical logic is utilised for predicting market conditions with sell spoofing orders.

10-minute re-estimation frequency						
Model	All Buy (%)	True Buy (%)	False Buy (%)	All Sell (%)	True Sell (%)	False Sell (%)
XGBoost	70.4	40.9	80.2	67.9	40.3	77.0
Random Forest	72.2	24.1	88.2	70.5	22.7	86.3
Decision Tree	65.9	43.5	73.2	63.6	45.2	69.7
RTSP	70.5	37.6	81.4	68.4	37.9	78.5
30-minute re-estimation frequency						
Model	All Buy (%)	True Buy (%)	False Buy (%)	All Sell (%)	True Sell (%)	False Sell (%)
XGBoost	70.6	41.6	79.8	67.7	41.1	76.6
Random Forest	72.1	24.1	87.9	70.4	22.8	86.2
Decision Tree	65.6	42.4	73.3	63.6	45.5	69.6
RTSP	70.2	37.0	81.2	68.3	38.1	78.2
60-minute re-estimation frequency						
Model	All Buy (%)	True Buy (%)	False Buy (%)	All Sell (%)	True Sell (%)	False Sell (%)
XGBoost	70.3	41.9	79.6	67.6	41.3	76.4
Random Forest	72.0	24.0	87.8	65.1	22.9	85.9
Decision Tree	65.0	42.9	72.3	63.2	45.7	69.0
RTSP	70.0	37.6	80.7	68.2	38.7	77.9

The RTSP indicates in real time the likelihood that the market conditions are advantageous for spoofers to place their orders. Out-of-sample forecasting demonstrates that a forecast combination outperforms individual forecasts. Table 13 indicates that Random Forest slightly surpasses the RTSP metric when evaluating all spoofing and non-spoofing market state forecasts. In a 10-minute re-estimation frequency, the RTSP measure significantly excels in predicting "True" spoofing orders, which is of paramount importance to regulators and traders as the ultimate users and benefactors of the developed algorithm. The objective of the machine learning application is to predict both the market conditions conducive to spoofing and those that are secure for trading without manipulative orders. RTSP enhances the predictions of individual machine learning algorithms, rendering the forecasting results reliable.

In comparing the forecasting accuracy of RTSP with that of other machine learning algorithms, it is evident that our developed methodology outperforms them in the specified setting. Table 15, illustrates the forecasting performance of alternative models, revealing lower results compared to the RTSP metric.

Table 15: Forecasting results of alternative models. The table presents 60-minute forecasting outcomes utilising alternative models, with the column *All correct* indicating the percentage of accurately predicted all orders, while the columns *True correct* and *False correct* represent the percentages of accurately predicted "True" and "False" spoofing orders, respectively.

Model	All correct (%)	True correct (%)	False correct (%)
Logistic Regression	57.7	37.4	65.7
Logistic Regression (Lasso)	57.9	37.6	65.9
Logistic Regression (Ridge)	57.7	37.4	65.7
K-nearest neighbors	58.5	38.2	66.5
Stochastic Gradient Descent	53.9	43.8	57.9
Support Vector Machines	29.9	17.5	34.8
LSTM with 10 layers	54.6	44.0	58.9
GRU	61.4	25.9	75.5
Perceptron	46.6	55.4	43.1

Finally, we demonstrate a potential application of the RTSP measure as a real-time risk indicator for authorities or exchanges, highlighting market conditions that may lead to the emergence of spoofing orders in the order book, resulting in a deteriorated market state. Figure 1 illustrates an instance of the graphical spoofing risk alert. We establish a critical value for the RTSP measure at 0.4, denoted by a horizontal black dotted line on the lower chart. RTSP surpasses the key value at the end of the trading day, approximately at 18:30. The upper chart illustrates numerous "True"

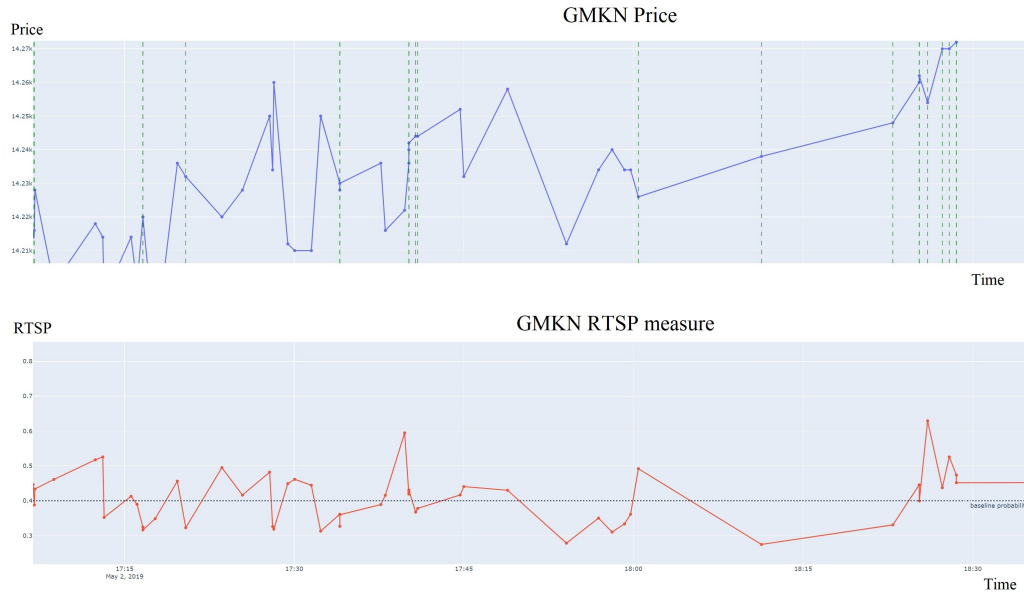


Figure 1: Implementation of RTSP. The figure illustrates an example of RTSP meatric as warning signals on GMKN stock. The vertical green dotted lines on the upper chart signify the occurrence of "True" spoofing orders. The horizontal black dotted line in the lower chart signifies a key threshold for the RTSP metric of 0.4.

suspicious spoofing orders at that time, shown by green dotted lines. Another event on the graph is the interval between 17:30 and 17:45 when RTSP markedly exceeds 0.4 and accurately predicts the placement of the spoofing order. The user can modify the critical value parameter and alter the signal display as a numerical indicator, warning, or trading feature on the selected platform. The RTSP metric could be integrated into trading platforms or surveillance systems. The approach is self-sufficient; no modifications are required to identify other types of manipulative strategies.

6. Conclusions

This research presents a data-driven methodology for the real-time identification and forecasting of market states associated with spoofing events. Our methodology is comprehensive, involving the identification of essential order book variables that can predict spoofing occurrences, alongside the selection of appropriate machine learning algorithms that integrate all vital data into a unified metric, the Real-Time Spoofing Probability (RTSP), designed to reflect the market’s vulnerability to intra-day manipulative strategies. State-of-the-art machine learning algorithms facilitate the analysis of data to identify the differences in the limit order book when suspected spoofers engage in trading vs when they do not. This adaptable methodology addresses non-linearities and variable interactions, while cross-validation and regularisation mitigate data overfitting. Our analysis presents compelling evidence for the model’s suitability for imbalanced datasets, exemplified by the real-time tick data utilised in our empirical investigation. This consistent performance indicates a much enhanced generalisation capacity relative to state-of-the-art models, making our method considerably more appealing to academics and regulators in financial institutions.

This empirical study’s primary contributions and its significant distinctions from existing literature can be encapsulated in three tiers. First, to the best of our knowledge, this work is the first to comprehensively apply a wide array of prevalent machine learning approaches and evaluate their predictive efficacy in forecasting periods of potential suspicious activities. Second, our methodology offers a thorough characterisation of the limit order book state by identifying pertinent financial microstructure indicators and underscores its pivotal role in uncovering fraudulent trading practices. Third, we focus on suspected spoofing incidents recognized by exchange authorities, ensuring replication of their spoofing identification methodology, albeit with a more constrained data sets.

Our approach facilitates the identification of intervals featuring potentially suspicious and fraudulent activities within the order book. Additionally, by leveraging our methodology, exchanges can proactively pinpoint periods with elevated risk of fraudulent activity, drawing from insights on past suspicious orders, typically identified retrospectively through comprehensive analysis of traders’ IDs and their historical trading strategies. By integrating our method into their surveillance systems, exchanges stand to achieve substantial enhancements in market quality, coupled with a notable reduction in instances of market manipulation.

This paper does not address whether incorporating market shocks, news, dividend activities, or

macroeconomic variables within our model could enhance predictive performance. In the future, we intend to do our study on an enhanced dataset comprising high-frequency tick data and lower-frequency macro-level data utilising a MIDAS technique. Furthermore, our architecture does not account for sequential aggressive spoofing orders. Nonetheless, the findings of this investigation furnish essential insights to regulators and exchanges to counteract artificial, manipulative orders, thereby enhancing overall market quality.

7. Appendices

7.1. Appendix 1. for variable selection

Table 16: Lasso regularisation for variable selection. This table presents the outcomes of the lasso regularisation method. We exclude from the analysis predictors with coefficient values below 0.001 (column *Coef.*).

Feature	Coef	Std.Err.	z	$P > z $	[0.025	0.975]
IMB	-0.3212	0.10522	-3.05267	0.002268	-0.52743	-0.11497
IMB_order	-0.00843	0.027257	-0.30911	0.75724	-0.06185	0.044997
FRA	-1.60787	0.144537	-11.1243	9.56E-29	-1.89116	-1.32458
FRB	0.880062	0.14531	6.056448	1.39E-09	0.595259	1.164864
QS	383.8753	34.00574	11.28854	1.49E-29	317.2252	450.5253
ES	122.714	49.63106	2.472524	0.013416	25.43888	219.9891
IMB_0	0.006483	0.014956	0.433459	0.664682	-0.02283	0.035795
IMB_1	0.097356	0.014714	6.616393	3.68E-11	0.068516	0.126196
IMB_2	0.077485	0.015252	5.080211	3.77E-07	0.047591	0.107378
IMB_3	0.078047	0.015426	5.059438	4.20E-07	0.047813	0.108282
IMB_4	-0.07434	0.015573	-4.77395	1.81E-06	-0.10487	-0.04382
IMB_5	-0.03868	0.01498	-2.58226	0.009816	-0.06804	-0.00932
UF_1ms_1	0.000422	0.000366	1.155149	0.24803	-0.00029	0.001139
UF_1ms_2	-0.00073	0.000496	-1.46944	0.141714	-0.0017	0.000243
UF_1ms_3	0.000657	0.000566	1.159823	0.246121	-0.00045	0.001766
UF_1ms_4	0.001032	0.000529	1.950864	0.051073	-4.80E-06	0.002068
UF_1ms_5	-0.00035	0.000502	-0.69271	0.48849	-0.00133	0.000636
UF_1ms_6	0.000565	0.000578	0.976952	0.328593	-0.00057	0.001698
UF_1ms_7	3.50E-05	0.000778	0.044997	0.96411	-0.00149	0.001559
UF_1ms_8	-0.00119	0.000701	-1.70177	0.088799	-0.00257	0.000181
UF_1ms_9	0.001496	0.000532	2.814629	0.004883	0.000454	0.002538
UF_1ms_10	-0.00107	0.000321	-3.32851	0.000873	-0.0017	-0.00044
UF_10ms_1	-0.00077	0.000741	-1.03415	0.301067	-0.00222	0.000686
UF_10ms_2	0.001162	0.000977	1.189622	0.234195	-0.00075	0.003076
UF_10ms_3	0.001405	0.001069	1.314504	0.188677	-0.00069	0.0035
UF_10ms_4	-0.00322	0.001053	-3.05752	0.002232	-0.00528	-0.00116
UF_10ms_5	0.003332	0.001106	3.012135	0.002594	0.001164	0.0055
UF_10ms_6	-0.00368	0.001174	-3.13422	0.001723	-0.00598	-0.00138
UF_10ms_7	0.002428	0.001261	1.924943	0.054237	-4.40E-05	0.0049
UF_10ms_8	0.001425	0.001161	1.227215	0.219742	-0.00085	0.0037
UF_10ms_9	-0.00373	0.000965	-3.86287	0.000112	-0.00562	-0.00184
UF_10ms_10	0.002529	0.000562	4.500931	6.77E-06	0.001428	0.00363
UF_50ms_1	0.007226	0.0018	4.015637	5.93E-05	0.003699	0.010754
UF_50ms_2	-0.00568	0.002153	-2.63767	0.008348	-0.0099	-0.00146
UF_50ms_3	0.000266	0.002007	0.132356	0.894703	-0.00367	0.004199
UF_50ms_4	0.000947	0.002137	0.443233	0.657597	-0.00324	0.005137
UF_50ms_5	-0.00443	0.00232	-1.90867	0.056305	-0.00897	0.000119
UF_50ms_6	0.01228	0.00243	5.053454	4.34E-07	0.007518	0.017043
UF_50ms_7	-0.0053	0.002431	-2.1814	0.029154	-0.01007	-0.00054
UF_50ms_8	-0.00452	0.001985	-2.27897	0.022669	-0.00841	-0.00063
UF_50ms_9	0.00126	0.001604	0.785475	0.432175	-0.00188	0.004405
UF_50ms_10	-0.001	0.001095	-0.91578	0.359781	-0.00315	0.001143
UF_100ms_1	-0.00551	0.001667	-3.30508	0.000949	-0.00878	-0.00224
UF_100ms_2	0.004442	0.002013	2.20594	0.027388	0.000495	0.008388
UF_100ms_3	-0.00302	0.001863	-1.61985	0.105264	-0.00667	0.000634
UF_100ms_4	0.002616	0.001967	1.330229	0.183443	-0.00124	0.006471
UF_100ms_5	0.000583	0.002108	0.276438	0.782211	-0.00355	0.004714
UF_100ms_6	-0.00897	0.002183	-4.11074	3.94E-05	-0.01325	-0.00469
UF_100ms_7	0.004389	0.002155	2.036936	0.041656	0.000166	0.008612
UF_100ms_8	0.001557	0.001819	0.856017	0.391988	-0.00201	0.005123
UF_100ms_9	0.004167	0.00153	2.722724	0.006475	0.001167	0.007166
UF_100ms_10	-0.0023	0.001034	-2.22182	0.026296	-0.00432	-0.00027
UF_600ms_1	-0.00046	0.000486	-0.93692	0.348798	-0.00141	0.000497

UF_600ms_2	0.000218	0.000583	0.37335	0.708888	-0.00092	0.00136
UF_600ms_3	0.001369	0.000599	2.283564	0.022397	0.000194	0.002544
UF_600ms_4	-0.00127	0.000594	-2.13249	0.032967	-0.00243	-0.0001
UF_600ms_5	0.000823	0.000608	1.35409	0.175708	-0.00037	0.002015
UF_600ms_6	0.000205	0.00062	0.330751	0.740833	-0.00101	0.001421
UF_600ms_7	-0.00108	0.000626	-1.7199	0.08545	-0.0023	0.00015
UF_600ms_8	0.001556	0.000608	2.559037	0.010496	0.000364	0.002748
UF_600ms_9	-0.00257	0.000598	-4.30355	1.68E-05	-0.00374	-0.0014
UF_600ms_10	0.00148	0.000354	4.175298	2.98E-05	0.000785	0.002175
VOL_1_1min	-332.465	112.7972	-2.94746	0.003204	-553.544	-111.387
VOL_1_2min	67.88061	179.9601	0.377198	0.706026	-284.835	420.596
VOL_1_5min	-55.8454	271.0023	-0.20607	0.836736	-587	475.3093
VOL_1_10min	36.52509	286.0571	0.127685	0.898399	-524.137	597.1868
VOL_2_1min	6.095202	1.104971	5.516166	3.46E-08	3.929499	8.260905
VOL_2_2min	-5.96658	1.590235	-3.75201	0.000175	-9.08338	-2.84978
VOL_2_5min	-4.02738	2.149298	-1.87381	0.060956	-8.23993	0.185166
VOL_2_10min	19.84205	2.331745	8.509528	1.75E-17	15.27192	24.41219
VOL_4_1min	2.754861	6194.337	0.000445	0.999645	-12137.9	12143.43
VOL_4_2min	2.870529	10254.95	0.00028	0.999777	-20096.5	20102.19
VOL_4_5min	2.577828	13808.9	0.000187	0.999851	-27062.4	27067.52
VOL_4_10min	3.352821	11427.39	0.000293	0.999766	-22393.9	22400.62
Hour	0.019725	0.002944	6.700296	2.08E-11	0.013955	0.025495
DistNorm50	20.25476	4455.851	0.004546	0.996373	-8713.05	8753.562
DistNorm20	-199.294	20273.16	-0.00983	0.992157	-39934	39535.38
DistNorm10	-194.514	18044.84	-0.01078	0.991399	-35561.7	35172.72
DistNormClean50	149.5656	27.31266	5.476056	4.35E-08	96.03381	203.0975
DistNormClean20	-176.587	31.28896	-5.64374	1.66E-08	-237.912	-115.262
DistNormClean10	93.17991	48.21986	1.932397	0.053311	-1.32927	187.6891
DistNormVol50	0.553286	0.078023	7.091356	1.33E-12	0.400364	0.706207
DistNormVol20	0.089309	0.030394	2.938428	0.003299	0.029739	0.14888
DistNormVol10	-0.01351	0.018983	-0.71187	0.476548	-0.05072	0.023692
DistNormVolClean50	-0.61615	0.09366	-6.57852	4.75E-11	-0.79972	-0.43258
DistNormVolClean20	-0.05694	0.036533	-1.55872	0.119063	-0.12855	0.014659
DistNormVolClean10	-0.08247	0.025564	-3.22598	0.001255	-0.13257	-0.03236
IMB_order_delta	-0.25393	0.188881	-1.3444	0.178818	-0.62413	0.116268
FRA_delta	-0.04651	0.375481	-0.12387	0.901414	-0.78244	0.689416
FRB_delta	-0.97608	0.377774	-2.58377	0.009773	-1.71651	-0.23566
QS_delta	70.65095	65.19404	1.083702	0.278497	-57.127	198.4289
IMB_order_delta_t2	0.366445	0.155094	2.362738	0.01814	0.062468	0.670423
FRA_delta_t2	-0.08441	0.315573	-0.26749	0.789091	-0.70293	0.534099
FRB_delta_t2	-0.36953	0.31831	-1.16091	0.245679	-0.99341	0.254347
QS_delta_t2	45.82324	57.36325	0.798826	0.424391	-66.6067	158.2531

Table 17: Elastic net regularisation for variable selection. This table presents the outcomes of the elastic net regularisation method with an alpha of 50%. We exclude from the study predictors with a coefficient value of zero. (column *Coef.*).

Features	Coef.	Features	Coef.
IMB	-0.0000020571	UF_100ms_7	-0.0000346457
IMB_order	0.0000011417	UF_100ms_8	-0.0000283937
FRA	-0.0000063929	UF_100ms_9	0.0000230656
FRB	-0.0000061548	UF_100ms_10	-0.0000084160
QS	0.0000000000	UF_600ms_1	-0.0000389521
ES	0.0000000000	UF_600ms_2	0.0000076768
IMB_0	0.0000007831	UF_600ms_3	0.0000759581
IMB_1	0.0000050115	UF_600ms_4	0.0000072271
IMB_2	0.0000032278	UF_600ms_5	-0.0001139139
IMB_3	0.0000030746	UF_600ms_6	-0.0000864398
IMB_4	-0.0000034290	UF_600ms_7	-0.0000397455
IMB_5	-0.0000029741	UF_600ms_8	0.0001119282
UF_1ms_1	0.0002037987	UF_600ms_9	0.0001552371
UF_1ms_2	0.0001268850	UF_600ms_10	0.0002367776
UF_1ms_3	0.0002629904	VOL_1_1min	0.0000000000
UF_1ms_4	0.0002236545	VOL_1_2min	0.0000000000
UF_1ms_5	0.0002090883	VOL_1_5min	0.0000000000
UF_1ms_6	0.0001877748	VOL_1_10min	0.0000000000
UF_1ms_7	0.0001086263	VOL_2_1min	0.0000001040
UF_1ms_8	-0.0000996961	VOL_2_2min	0.0000000555
UF_1ms_9	-0.0001278121	VOL_2_5min	0.0000001025
UF_1ms_10	-0.0002366476	VOL_2_10min	0.0000001424
UF_10ms_1	0.0001470780	VOL_4_1min	0.0000000000
UF_10ms_2	0.0000016729	VOL_4_2min	0.0000000000
UF_10ms_3	0.0000927591	VOL_4_5min	0.0000000000
UF_10ms_4	0.0000683550	VOL_4_10min	0.0000000000
UF_10ms_5	0.0001042563	Hour	-0.0000807383
UF_10ms_6	0.0001729422	DistNorm50	-0.0000000461
UF_10ms_7	0.0001453083	DistNorm20	-0.0000000167
UF_10ms_8	0.0000134912	DistNorm10	-0.0000000069
UF_10ms_9	0.0000455122	DistNormClean50	-0.0000000466
UF_10ms_10	0.0000721976	DistNormClean20	-0.0000000175
UF_50ms_1	0.0000791259	DistNormClean10	-0.0000000077
UF_50ms_2	-0.0000835425	DistNormVol50	0.0000051493
UF_50ms_3	-0.0000380171	DistNormVol20	0.0000037754
UF_50ms_4	-0.0000731352	DistNormVol10	0.0000003867
UF_50ms_5	-0.0001232195	DistNormVolClean50	0.0000038100
UF_50ms_6	-0.0000435570	DistNormVolClean20	0.0000026168
UF_50ms_7	-0.0000991683	DistNormVolClean10	-0.0000005011
UF_50ms_8	-0.0001743680	IMB_order_delta	-0.0000000849
UF_50ms_9	-0.0001484642	FRA_delta	0.0000000824
UF_50ms_10	-0.0001885770	FRB_delta	-0.0000001857
UF_100ms_1	0.0000356753	QS_delta	0.0000000000
UF_100ms_2	-0.0000570796	IMB_order_delta_t2	0.0000001432
UF_100ms_3	-0.0000136687	FRA_delta_t2	0.0000000255
UF_100ms_4	-0.0000411564	FRB_delta_t2	-0.0000001150
UF_100ms_5	-0.0001139238	QS_delta_t2	0.0000000000
UF_100ms_6	-0.0000416980		

7.2. Appendix 2. Correlation matrix of predictors

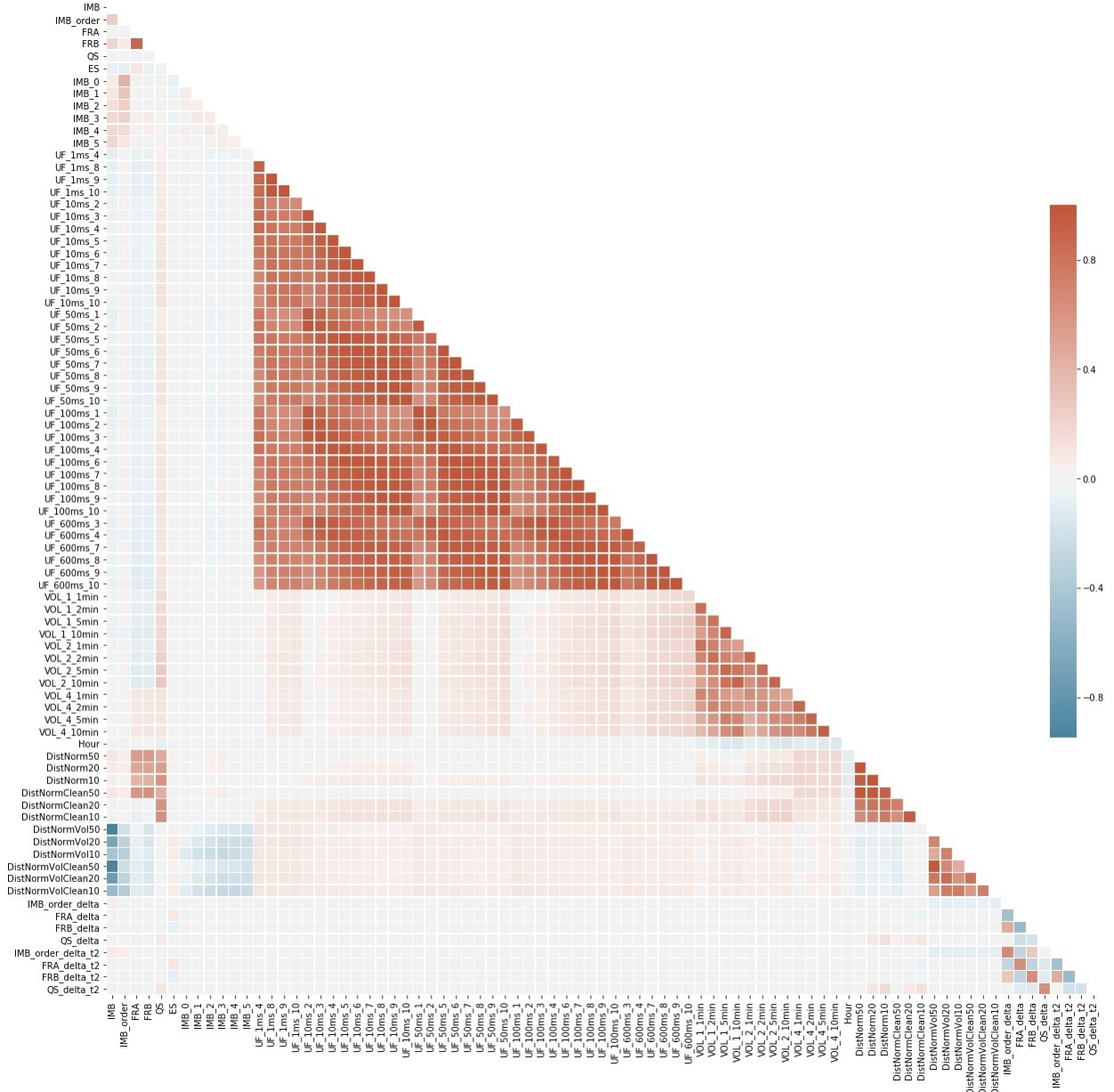


Figure 2: Correlation matrix of predictors

7.3. Appendix 3. List of features

We exclude unnecessary features from subsequent analysis utilising the lasso and elastic net variable selection method with coefficients set to 0. The subsequent features are unnecessary: QS, ES, VOL_1_1min, VOL_1_2min, VOL_1_5min, VOL_1_10min, VOL_3_1min, VOL_3_2min, VOL_3_5min, VOL_3_10min, QS_delta, QS_delta_t2. The variable 'Hour' contributes to model overfitting due to the absence of "False" orders during the first and last trading hours. The 'VOL_2_1min' variable contains numerous unidentified values. Consequently, we eliminate the two features 'Hour' and 'VOL_2_1min'.

The ultimate selection of predictors for subsequent analysis is as follows: IMB, IMB_order, FRA, FRB, IMB_0, IMB_1, IMB_2, IMB_3, IMB_4, IMB_5, UF_1ms_4, UF_1ms_8, UF_1ms_9, UF_1ms_10, UF_10ms_2, UF_10ms_3, UF_10ms_4, UF_10ms_5, UF_10ms_6, UF_10ms_7, UF_10ms_8, UF_10ms_9, UF_10ms_10, UF_50ms_1, UF_50ms_2, UF_50ms_5, UF_50ms_6, UF_50ms_7, UF_50ms_8, UF_50ms_9, UF_50ms_10, UF_100ms_1, UF_100ms_2, UF_100ms_3, UF_100ms_4, UF_100ms_6, UF_100ms_7, UF_100ms_8, UF_100ms_9, UF_100ms_10, UF_600ms_3, UF_600ms_4, UF_600ms_7, UF_600ms_8, UF_600ms_9, UF_600ms_10, VOL_2_2min, VOL_2_5min, VOL_2_10min, DistNorm50, DistNorm20, DistNorm10, DistNormClean50, DistNormClean20, DistNormClean10, DistVol50, DistVol20, DistVol10, DistVolClean50, DistVolClean20, DistVolClean10, IMB_delta, FRA_delta, FRB_delta, IMB_delta_t2, FRA_delta_t2, FRB_delta_t2.

7.4. Appendix 4. Diebold-Mariano test

Table 18: Diebold-Mariano test. The table presents the test results that compare the predictive qualities of the models in pairs. We evaluate Random Forest, Decision Tree, and XGBoost against other machine learning models. The null hypothesis posits that the models are equivalent. If the p-value is below 0.05, the models are statistically unequal. We denote the findings with a '-' sign if the first model outperforms the second model; conversely, a '+' sign indicates the reverse.

First model	Second model	Diebold-Mariano statistic	P-value
Random Forest	Logistic regression	-54.09	0.00
Random Forest	SVM	-48.59	0.00
Random Forest	KNN	-24.51	0.00
Random Forest	Naive Bayes	-62.61	0.00
Random Forest	SGD	-59.31	0.00
Random Forest	Decision Tree	-22.97	0.00
Random Forest	Gradient Boosting	-26.32	0.00
Random Forest	XGBoost	-26.81	0.00
Random Forest	LSTM with 10 layers	-50.05	0.00
Random Forest	GRU	-52.20	0.00
Random Forest	Perceptron	-54.35	0.00
Decision Tree	Logistic regression	-37.09	0.00
Decision Tree	SVM	-26.91	0.00
Decision Tree	KNN	-1.83	0.07
Decision Tree	Naive Bayes	-37.16	0.00
Decision Tree	SGD	-36.24	0.00
Decision Tree	Gradient Boosting	-1.66	0.10
Decision Tree	XGBoost	-2.12	0.03
Decision Tree	LSTM with 10 layers	-33.51	0.00
Decision Tree	GRU	-37.10	0.00
Decision Tree	Perceptron	-34.42	0.00
XGBoost	Logistic regression	-33.59	0.00
XGBoost	SVM	-25.31	0.00
XGBoost	KNN	0.25	0.80
XGBoost	Naive Bayes	-39.49	0.00
XGBoost	SGD	-36.87	0.00
XGBoost	Gradient Boosting	1.63	0.10
XGBoost	LSTM with 10 layers	-29.46	0.00
XGBoost	GRU	-32.16	0.00
XGBoost	Perceptron	-33.03	0.00

7.5. Appendix 5. Model Confidence Set test

Table 19: Model Confidence Set test. The table presents the execution of the Model Confidence Set (MCS) methodology for model comparison at the 95% confidence level (α is 0.05) utilising the mean absolute error (MAE) loss function. Panel A indicates that among all the models employed in the research, Random Forest is the superior model. Panel B presents the results of the model comparison test, omitting Random Forest, to identify the second-best model.

Panel A							
Model	Elimination result						
Logistic regression	eliminated						
SGD	eliminated						
GRU	eliminated						
Naive Bayes	eliminated						
LSTM with 10 layers	eliminated						
SVM	eliminated						
XGBoost	eliminated						
Gradient Boosting	eliminated						
KNN	eliminated						
Random Forest	eliminated						
Superior Set Model created	Rank_M	v_M	MCS_M	Rank_R	v_R	MCS_R	Loss
Random Forest	1	-28.06988	1	1	-28.06988	1	0.2332108
p-value : [1] 0							
Panel B							
Model	Elimination result						
Logistic regression	eliminated						
Naive Bayes	eliminated						
GRU	eliminated						
SGD	eliminated						
LSTM with 10 layers	eliminated						
SVM	eliminated						
Superior Set Model created	Rank_M	v_M	MCS_M	Rank_R	v_R	MCS_R	Loss
KNN	3	0.6730877	0.769	3	1.886552	0.212	0.3200074
Decision Tree	1	-2.3702309	1.000	1	-1.876958	1.000	0.3126542
Gradient Boosting	2	0.6042045	0.813	2	1.876958	0.223	0.3192163
XGBoost	4	1.7018618	0.159	4	2.368780	0.081	0.3210779
p-value : [1] 0.159							

7.6. Appendix 5. Parameter optimisation for machine learning models

To enhance the selected models, we adjust the parameters while training on 75% of the data and evaluating on 25% of the data. Increasing 'max_depth' to 41 and 'n_estimators' to 600 enhances the accuracy of the Random Forest model. Increasing 'max_depth' further does not enhance the findings, but augmenting 'n_estimators' to 1000 yields a 0.5% improvement in accuracy; nevertheless, this significantly escalates model complexity, resulting in diminished performance. For XGBoost, we select 'n_estimators' as 500, since increasing it to 1000 yields just a 1.5% improvement in model accuracy, while augmenting complexity and reducing performance speed. Conversely, reducing 'n_estimators' to 100 results in a substantial accuracy decline of 5%. Reducing 'max_depth' below 10 leads to a decline in accuracy. For the Decision Tree, a 'max_depth' above 5 does not produce favourable outcomes; similarly, augmenting 'min_samples_leaf' beyond 3 fails to enhance results.

Consequently, we selected the following parameters for the chosen machine learning models.

Parameters of the Random Forest model:

```
clf = RandomForestClassifier (random_state = 0
n_estimators = 600, min_samples_split = 5,
min_samples_leaf = 1, max_features = 'sqrt',
max_depth = 41, bootstrap = False)
```

Parameters of the XGBoost model:

```
clf = xgb.XGBClassifier (learning_rate = 0.01,
n_estimators = 500,
max_depth = 10,
min_child_weight = 1,
gamma = 0.1,
subsample = 0.8,
colsample_bytree = 0.8,
objective = 'binary : logistic',
nthread = 4,
scale_pos_weight = 1.5, random_state = 0)
```

Parameters of the Decision model:

```
clf = DecisionTreeClassifier
(random_state = 0, max_depth = 5, min_samples_leaf = 3)
```

7.7. Appendix 6. Illustration of expanding and rolling validation approaches

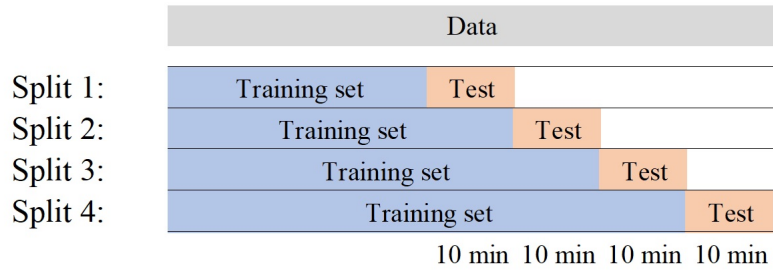


Figure 3: Illustration of the expanding validation approach

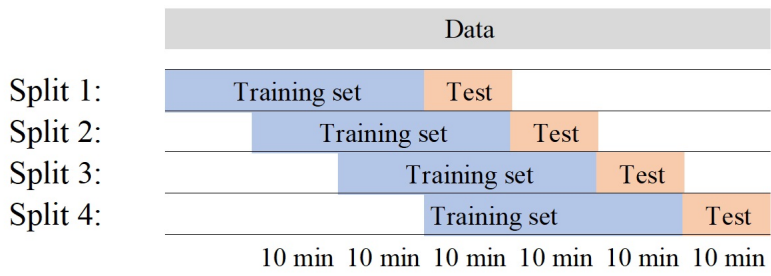


Figure 4: Illustration of the rolling validation approach

References

- M. Punniyamoorthy, J. J. Thoppan, Ann-ga based model for stock market surveillance, *Journal of Financial Crime* (2013).
- S. Imisiker, B. Tas, Which firms are more prone to stock market manipulation?, *Emerging Markets Review* 16 (2013) 119–130.
- L. Guiso, P. Sapienza, L. Zingales, Trusting the stock market, the *Journal of Finance* 63 (2008) 2557–2600.
- Dodd-Frank, Dodd-frank wall street reform and consumer protection act, 2010.
- M. J. Aitken, F. H. de B Harris, S. Ji, A worldwide examination of exchange market quality: Greater integrity increases market efficiency, *Journal of Business Ethics* 132 (2015) 147–170.
- D. Cumming, S. Johan, D. Li, Exchange trading rules and stock market liquidity, *Journal of Financial Economics* 99 (2011) 651–671.
- H. Ögüt, M. M. Doğanay, R. Aktaş, Detecting stock-price manipulation in an emerging market: The case of turkey, *Expert Systems with Applications* 36 (2009) 11944–11949.
- K. Golmohammadi, O. R. Zaiane, Time series contextual anomaly detection for detecting market manipulation in stock market, in: 2015 IEEE international conference on data science and advanced analytics (DSAA), IEEE, 2015, pp. 1–10.
- Y. Cao, Y. Li, S. Coleman, A. Belatreche, T. M. McGinnity, Detecting price manipulation in the financial market, in: 2014 IEEE Conference on Computational Intelligence for Financial Engineering & Economics (CIFEr), IEEE, 2014, pp. 77–84.
- J.-N. Tuccella, P. Nadler, O. Şerban, Protecting retail investors from order book spoofing using a gru-based detection model, *arXiv:2110.03687* (2021).
- X. Tao, A. Day, L. Ling, S. Drapeau, On detecting spoofing strategies in high-frequency trading, *Quantitative Finance* 22 (2022) 1405–1425.
- Y. Cao, Y. Li, S. Coleman, A. Belatreche, T. M. McGinnity, Adaptive hidden markov model with anomaly states for price manipulation detection, *IEEE transactions on neural networks and learning systems* 26 (2015) 318–330.
- B. Williams, A. Skrzypacz, Spoofing in equilibrium, Available at SSRN (2020).
- Á. Cartea, R. Payne, J. Penalva, M. Tapia, Ultra-fast activity and intraday market quality, *Journal of Banking & Finance* 99 (2019) 157–181.
- C. M. Lee, Market integration and price execution for nyse-listed securities, *The Journal of Finance* 48 (1993) 1009–1038.
- M. E. Blume, M. A. Goldstein, Displayed and effective spreads by market, Rodney L. White Center for Financial Research Working Paper (1992).
- M. Aitken, A. Frino, The determinants of market bid ask spreads on the australian stock exchange: Cross-sectional analysis, *Accounting & Finance* 36 (1996) 51–63.
- J. F. Egginton, B. F. Van Ness, R. A. Van Ness, Quote stuffing, *Financial Management* 45 (2016) 583–608.
- J. Hasbrouck, G. Saar, Low-latency trading, *Journal of Financial Markets* 16 (2013) 646–679.
- A. Cartea, S. Jaimungal, Y. Wang, Spoofing and price manipulation in order-driven markets, *Applied Mathematical Finance* 27 (2020) 1–2, 67–98.
- A. Ranaldo, Order aggressiveness in limit order book markets, *Journal of Financial Markets* 7 (2004) 53–74.
- B. Biais, P. Hillion, C. Spatt, An empirical analysis of the limit order book and the order flow in the paris bourse, the *Journal of Finance* 50 (1995) 1655–1689.
- Y. Cao, Y. Li, S. Coleman, A. Belatreche, T. McGinnity, Adaptive hidden markov model with anomaly states for price manipulation detection, *IEEE Transactions on Neural Networks and Learning Systems* 26 (2015) 318–330.
- R. Tibshirani, Regression shrinkage and selection via the lasso, *Journal of the Royal Statistical Society: Series B (Methodological)* 58 (1996) 267–288.
- T. Hastie, R. Tibshirani, J. H. Friedman, J. H. Friedman, The elements of statistical learning: data mining, inference, and prediction, volume 2, Springer, 2009.
- H. Zou, T. Hastie, Regularization and variable selection via the elastic net, *Journal of the royal statistical society: series B (statistical methodology)* 67 (2005) 301–320.
- S. Khodabandehlou, S. A. H. Golpayegani, Market manipulation detection: A systematic literature review, *Expert Systems with Applications* 210 (2022) 118330.
- Z.-H. Zhou, Ensemble methods: foundations and algorithms, CRC press, 2012.
- T. K. Ho, Random decision forests, in: Proceedings of 3rd international conference on document analysis and recognition, volume 1, IEEE, 1995, pp. 278–282.
- L. Breiman, Random forests, *Machine learning* 45 (2001) 5–32.

- T. Hastie, R. Tibshirani, J. H. Friedman, J. H. Friedman, The elements of statistical learning: data mining, inference, and prediction, Springer, 2017.
- T. Chen, T. He, M. Benesty, V. Khotilovich, Y. Tang, H. Cho, K. Chen, R. Mitchell, I. Cano, T. Zhou, et al., Xgboost: extreme gradient boosting, R package version 0.4-2 1 (2015) 1–4.
- G. Claeskens, J. R. Magnus, A. L. Vasnev, W. Wang, The forecast combination puzzle: A simple theoretical explanation, International Journal of Forecasting 32 (2016) 754–762.